

**Algoritmo de Cifrado de Una Vía
Informe**

**Juan David Romero Morales – 20222020102
Jaider Alejandro González Rojas - 20242020075
Jimmy Alejandro Granados Becerra - 20251020034
Juan Sebastian Gutiérrez Poveda - 20251020199**

**Universidad Distrital Francisco Jose de Caldas
Ciencias de la Computación II – Grupo 020-84
Docente: Simar Enrique Herrera**

**Bogotá D.C.
15 De Septiembre De 2026**

Tabla de Contenido

	Pág.
1. Introducción	3
2. Objetivos	4
2.1. Objetivo General.....	4
2.2. Objetivos Específicos	4
3. Descripción y Funcionamiento del Algoritmo.....	5
3.1. Función de los componentes.....	5
3.2. Características del algoritmo determinístico.....	6
4. Explicación del Código Desarrollado	7
4.1. Correspondencia matemática con el lenguaje Java.....	7
4.2. Algoritmo de mezcla acumulativa y relleno de caracteres.....	7
4.3. Implementación de difusión de bits y prevención de secuencias de ceros	8
5. Análisis de Seguridad y Limitaciones	9
5.1. Vulnerabilidades y limitaciones de la función hash educativa	9
5.2. Comparación técnica con el estándar SHA-256.....	9
6. Pruebas.....	11
6.1. Trazabilidad y ejemplo de ejecución (Cadena "ABC")	11
6.2. Verificación de salidas hexadecimales de 256 bits	11
7. Conclusión	13

1. Introducción

LINK DEL REPOSITORIO: https://github.com/Jimmy86gb/CC2_GR5_020_84.git

Ruta de Entrega: 1ER_CORTE/4_Cifrado_De_Una_Sola_Via/Entrega

El presente documento detalla la implementación, estructuración y análisis de una función hash educativa de 256 bits. El desarrollo de este tipo de algoritmos desde cero permite estudiar y comprender desde adentro conceptos fundamentales de las ciencias de la computación, tales como el determinismo, la mezcla y difusión de datos, la dependencia del orden temporal de los caracteres y la generación de colisiones estructurales.

Conservando una fórmula basada en operaciones lógicas XOR y dos estados numéricos acumulativos, el algoritmo propuesto da el salto a un espacio de estado mucho más amplio empleando la clase BigInteger del lenguaje Java, lo que le permite operar fluidamente bajo un módulo de 2^{256} . De este modo, se emula la longitud de salida de estándares modernos. Sin embargo, es de vital importancia destacar que este desarrollo tiene propósitos estrictamente pedagógicos: no incluye rondas complejas de compresión ni sustitución criptográfica, por lo que no sustituye a una función hash real ni debe utilizarse para proteger contraseñas, tokens de sesión o datos sensibles en entornos de producción.

2. Objetivos

2.1. Objetivo General

Comprender la estructura y comportamiento de una función hash de 256 bits mediante una implementación educativa en Java.

2.2. Objetivos Específicos

- **Analizar el funcionamiento interno:** Estudiar detalladamente la mezcla acumulativa y las operaciones matemáticas de dispersión utilizando la clase inmutable BigInteger para superar los límites de la memoria de los tipos de datos primitivos.
- **Evaluar las propiedades del hash:** Comprobar empíricamente las características determinísticas del algoritmo, así como su sensibilidad matemática frente a variaciones mínimas en la longitud, el contenido y el orden de la entrada de texto.
- **Identificar debilidades estructurales:** Reconocer las vulnerabilidades y limitaciones teóricas de un algoritmo de hash de diseño lineal con propósitos puramente académicos, contrastándolo frente a los estrictos lineamientos de seguridad de estándares de la industria como SHA-256.

3. Descripción y Funcionamiento del Algoritmo

El algoritmo inicia su ciclo de ejecución guardando la longitud original del texto introducido antes de proceder con cualquier manipulación de la cadena. Este paso inicial es crucial, ya que permite al sistema diferenciar matemáticamente entradas que podrían ser visualmente idénticas pero que varían únicamente por llevar espacios finales. Posteriormente, inicializa dos estados fundamentales: la variable penúltima con un valor estático inicial de 3, y la variable anterior con el resultado dinámico de la operación 7 XOR la longitud original.

Con el fin de garantizar una cantidad mínima de iteraciones matemáticas (incluso para textos muy cortos o vacíos), si la entrada introducida tiene menos de 32 caracteres, el algoritmo la completa añadiendo espacios en blanco (carácter ASCII 32) hasta alcanzar dicho umbral.

Durante la fase de recorrido del texto, por cada iteración se calcula una variable temporal denominada mezcla. Esta se obtiene aplicando la operación XOR entre el valor numérico (ASCII) del carácter procesado y su posición exacta en la cadena multiplicada por 9. El estado nuevo se calcula entonces realizando una combinación lineal y lógica: el estado anterior multiplicado por 12, el penúltimo multiplicado por 5, y el valor de la mezcla. A esta amalgama de datos se le aplica finalmente el módulo 2^{256} para evitar el desbordamiento infinito. Al concluir cada ciclo, los estados acumulativos se desplazan (el penúltimo toma el valor del anterior, y el anterior asume el nuevo valor calculado).

3.1. Función de los componentes

- **BigInteger:** Es el pilar de la representación de datos. Permite almacenar, desplazar y operar fluidamente con estados gigantes de 256 bits sin chocar con el límite convencional de 64 bits que posee el tipo de dato primitivo long nativo de Java.
- **Longitud original:** Influye de manera directa en la "semilla" o estado inicial antes del proceso de relleno, sirviendo como métrica de distinción para textos con espacios finales añadidos.
- **Operación XOR:** Operador a nivel de bits (O exclusivo) cuya función es combinar la información binaria de sus respectivos operandos de manera irreversible sin acarreos aritméticos, un principio básico en la criptografía.
- **Posición por 9:** Actúa como factor de dependencia posicional. Garantiza que un mismo carácter se mezcle matemáticamente de forma distinta dependiendo de la posición específica que ocupa, evitando que los anagramas (ej. "AMOR" y "ROMA") generen el mismo hash.
- **Multiplicadores 12 y 5:** Otorgan un tratamiento de escalamiento diferencial y no lineal a los estados anterior y penúltimo instantes antes de procesar el XOR, amplificando las pequeñas diferencias en cada ronda.

- **Relleno (Padding):** Asegura que las entradas muy cortas procesen obligatoriamente un bloque mínimo de 32 caracteres, forzando al algoritmo a realizar al menos 32 rondas de mezcla para diluir la entropía inicial.
- **Módulo 2^{256} :** Mantiene constantemente cada estado numérico truncado dentro del espacio delimitado, garantizando que el valor fluctúe cíclicamente en el rango de 0 a $2^{256} - 1$.
- **Hexadecimal:** Modifica únicamente la representación visual del número final al usuario. Es indispensable aclarar que mostrar los datos en base 16 no le suma ningún tipo de robustez o seguridad matemática al algoritmo.

3.2. Características del algoritmo determinístico

- **Determinística:** Es la regla de oro del hash. Se garantiza absoluta e incondicionalmente que una misma entrada exacta siempre producirá la misma salida sin importar cuántas veces se ejecute.
- **Dependiente del orden:** Cambiar la posición u orden de un solo carácter altera inmediatamente la variable temporal de mezcla y, en consecuencia, provoca un efecto dominó que transforma todos los estados posteriores.
- **Sensible a la longitud original:** Entradas como "ABC" y "ABC " (con un espacio extra) inician su ciclo de transformaciones con variables de estado anterior totalmente distintas desde la iteración cero.
- **Procesamiento mínimo fijo:** Cualquier entrada corta recorre de forma predeterminada 32 posiciones, homogeneizando el tiempo de ejecución mínimo.
- **Salida de 256 bits:** El resultado computado se presenta estandarizado con una longitud de exactamente 64 dígitos en base hexadecimal, emulando la presentación visual de familias de algoritmos como SHA-2.
- **Con colisiones estructurales:** Debido al Principio del Palomar de las matemáticas, al mapear una cantidad infinita de textos de entrada posibles hacia un espacio finito de salidas (por muy grande que sea 2^{256}), es un hecho comprobable que distintas entradas llegarán inevitablemente a producir el mismo resultado (colisión).

4. Explicación del Código Desarrollado

4.1. Correspondencia matemática con el lenguaje Java

Dentro de la estructura de programación orientada a objetos de Java, la expresión matemática constante de límite superior se construye utilizando `BigInteger.ONE.shiftLeft(256)`. Esta instrucción toma un bit en la posición 1 y lo desplaza (Shift) exactamente 256 posiciones lógicas hacia la izquierda, generando matemáticamente el valor 2^{256} .

La conversión de variables primitivas convencionales de tipo `int` o `long` a objetos operables se realiza mediante el método de fábrica `BigInteger.valueOf`. Las multiplicaciones aritméticas no utilizan operadores nativos (`*`), sino que se ejecutan invocando el método `.multiply()`. Por ejemplo, la invocación de `anterior.multiply(MULTIPLICADOR_ANTERIOR)` representa de forma programática el cálculo matemático $anterior \times 12$. Las demás operaciones de la fórmula planteada se traducen de forma directa encadenando los métodos `.xor()` y `.mod()`.

Es de suma relevancia mencionar que la clase `BigInteger` en Java es de naturaleza inmutable; por lo cual, cada operación matemática de la fórmula no modifica las variables existentes en memoria, sino que el motor de Java instancia y retorna un objeto de memoria totalmente nuevo por cada cálculo realizado en el bucle.

4.2. Algoritmo de mezcla acumulativa y relleno de caracteres

A nivel de control de flujo, el código fuente define la constante `LONGITUD_MINIMA` con el valor de 32. El método auxiliar `rellenarHasta32Caracteres` implementa un ciclo precondicional tipo `while` diseñado para inyectar espacios (carácter ' ', cuyo valor ASCII es 32) al texto ingresado por el usuario, hasta lograr emparejar la longitud de la cadena con el límite mínimo estipulado.

Durante el procesamiento de la información en la iteración principal (bucle `for`), se evalúa progresivamente cada índice. El objetivo de este bucle es extraer el carácter en curso (`charAt`), obtener su posición indexada e identificar su peso numérico (`valorCaracter`). Tras recolectar estos insumos, el programa procede a instanciar y calcular las variables locales de tipo `BigInteger` denominadas `mezcla` y `nuevo`. En las líneas finales de la iteración, se produce un desplazamiento temporal clásico donde se reemplazan progresivamente los objetos referenciados por `penúltimo` y `anterior`, preparándolos para la siguiente ronda del bucle.

4.3. Implementación de difusión de bits y prevención de secuencias de ceros

Un problema común en algoritmos de diseño simple es la falta del "efecto avalancha" (donde un pequeño cambio en la entrada debería cambiar aproximadamente el 50% de los bits de salida). Para maximizar la dispersión final del resultado y tratar de subsanar esta debilidad, el código incorpora métodos finales adicionales post-procesamiento.

En primer lugar, la función `difundirBits` distribuye la entropía acumulada uniformemente entre los 256 bits resultantes usando operaciones lógicas de desplazamiento hacia la derecha (`shiftRight(64)` y `shiftRight(128)`) combinadas con un XOR. Estas operaciones se multiplican por una `CONSTANTE_DIFUSION`. Esta constante (cuyo valor en base 16 es `9E3779B97F4A7C15F39CC0605CEDC8341082276BF3A27251F86C6A11D0C18E95`) tiene raíces en la razón áurea matemática, un número irracional ampliamente utilizado en ciencias de la computación (por ejemplo, en el algoritmo TEA) para romper simetrías indeseadas y dispersar bits uniformemente.

En segundo lugar, se emplea el método iterativo `eliminarSecuenciaCeros`. Este método busca evitar salidas visualmente predecibles u homogéneas. Aplica un bucle `while` que supervisa continuamente si la representación hexadecimal en formato `String` del hash contiene el patrón anómalo de seis ceros consecutivos `"000000"`. De detectarlo, el sistema altera drásticamente el estado numérico aplicando una operación XOR junto a una constante estática de `PERTURBACION` (cuyo valor está conformado por una extensa cadena de segmentos en hexadecimal `A5`, equivalente al patrón binario alternante `10100101`).

Seguido de esto, se aplica el módulo correspondiente, y el programa continúa ciclándose automáticamente hasta purgar por completo dicho patrón de la salida final, asegurando mayor calidad visual en el hash resultante.

5. Análisis de Seguridad y Limitaciones

5.1. Vulnerabilidades y limitaciones de la función hash educativa

La función diseñada a lo largo de este proyecto es de carácter estrictamente educativo y descriptivo; no posee grado alguno de robustez criptográfica. El simple hecho de aumentar el espacio a 256 bits no corrige por sí solo sus debilidades matemáticas estructurales. A diferencia de las funciones seguras de la actualidad, el núcleo de nuestro algoritmo carece de funciones de compresión complejas y carece del uso de tablas de constantes pseudoaleatorias por ronda.

Las constantes multiplicadoras (12 y 5) y las operaciones del código son completamente lineales, públicas y de fácil comprensión de ingeniería inversa, lo que permite que atacantes puedan formular ataques matemáticos y probar entradas por fuerza bruta a gran velocidad. Sumado a ello, al ser una función pura, el algoritmo no utiliza *salt* criptográfico (salteo aleatorio), por lo que los textos o contraseñas que son idénticos siempre producirán hashes iguales, facilitando los ataques de tablas arcoíris (*Rainbow Tables*). A nivel matemático y probabilístico, las colisiones continúan siendo altamente probables bajo criptoanálisis diferencial debido a que el grado de difusión bit a bit por cada ronda es bajo. Esta implementación no ha sido analizada formalmente ni sometida a pruebas estandarizadas (como las de NIST) para resistir ataques de preimagen, ataques de segunda preimagen ni ataques de colisión pura. En ninguna circunstancia del mundo real debe emplearse este fragmento de código para proteger contraseñas en bases de datos, validación de firmas digitales, sistemas de control de versiones o protección de información en entornos web.

5.2. Comparación técnica con el estándar SHA-256

Aspecto	Función propuesta	SHA-256
Salida	256 bits y 64 caracteres hexadecimales	256 bits y 64 caracteres hexadecimales
Objetivo	Aprendizaje	Seguridad criptográfica
Estructura	XOR, posición, tamaño de longitud y dos estados acumulativos	64 rondas integradas a operaciones criptográficas

Aspecto	Función propuesta	SHA-256
Relleno	Espacios simples hasta 32 caracteres	Relleno binario normado y definido por el estándar
Seguridad	No analizada formalmente	Rigurosamente diseñado y analizado
Manejo de Contraseñas	No sirve para esto	No de forma directa; se prefieren Argon2, bcrypt o scrypt

6. Pruebas

6.1. Trazabilidad y ejemplo de ejecución (Cadena "ABC")

Al evaluar el ingreso de la cadena de texto "ABC", se detecta que su longitud original es de 3. Gracias a esto, el estado penúltimo arranca en 3 y el anterior arranca en el valor 4 (derivado del cálculo $7 \text{ XOR } 3$). A continuación, el programa agrega automáticamente 29 espacios para completar las 32 posiciones exigidas. La tabla teórica de trazabilidad para este escenario muestra cómo evolucionan los valores:

Iteración	Carácter	Valor	Mezcla	Nuevo valor decimal
1	A	65	72	119
2	B	66	80	1488
3	C	67	88	18379
4	espacio	32	4	213136
31	espacio	32	311	28852585487095 07439401900920 6056877
32	espacio	32	256	33645361348804 59422650720226 66820762

6.2. Verificación de salidas hexadecimales de 256 bits

La implementación escrita en Java incluye el método estático `convertirAHexadecimal`, el cual asegura, a través del formateador de cadenas `%064X`, que la cifra se proyecte correctamente. El formato resultante comprende exactamente 64 dígitos correspondientes a 256 bits que se dividen entre 4 bits por cada uno de los dígitos hexadecimales, abarcando desde 64 ceros hasta 64 letras F. Retomando la evaluación de la entrada "ABC" (previo a aplicar las operaciones finales dinámicas del código final), el Hash Hexadecimal puro genera:

[illegible]

Tras la ejecución completa de la clase Mezclador256Bits y sus algoritmos de entropía, se visualiza mediante consola tanto la estructura del Hash decimal completo como su salida final depurada en Hexadecimal.

7. Conclusión

El diseño e implementación de esta versión escalada de 256 bits conserva íntegramente la lógica educativa de su predecesora, logrando adaptar efectivamente su representación y capacidad de memoria aprovechando el uso robusto de la clase BigInteger de Java. El algoritmo cumple con creces los objetivos pedagógicos planteados: discrimina apropiadamente los ingresos mediante la lectura temprana de la longitud original, asegura su nivel de procesamiento mínimo iterando obligatoriamente 32 posiciones para expandir la cadena, y estabiliza el margen numérico general aplicando un truncamiento constante con el módulo 2^{256} .

Las fases de pre y post-procesamiento (incluyendo la inyección de espacios, la constante de difusión áurea y la prevención dinámica de cadenas de ceros) añaden valiosas capas de complejidad técnica que enriquecen el aprendizaje sobre cómo mitigar sesgos en los datos generados. En resumen, este desarrollo sirve para estudiar de manera totalmente transparente y manual cómo interactúan a bajo nivel los estados acumulativos, las operaciones de bits XOR, los formatos numéricos grandes en lenguajes de tipado estricto y cómo se provocan las colisiones estructurales. No obstante, se concluye reiterando firmemente que este tipo de lógicas ad-hoc no deben confundirse jamás con el rigor matemático de los estándares avalados como SHA-256, ni deben ser aprovechados como mecanismos válidos para entornos reales de seguridad informática.